

目录

§ 1 lua 类型 1

1.1 字符相关 1

1.2 索引条目 9

1.3 实用类型 13

1.4 PCRE2 正则表达式 15

§ 2 可加载的外部库 18

2.1 cindex.table 18

2.2 icu.table 19

2.3 digest 哈希算法 19

2.4 LPeg 19

§ 3 代码索引 20

§ 1 lua 类型

1.1 字符相关

一般情况下，cindex 中保存数据的类型都能使用 == nil 来判断它是否为无效值。

```
Char(number) → Char;
Char(Char) → Char;
Char(string) → Char | nil;
```

Char

该类型的构造器。返回一个 Char，这是 UTF-32 编码的 Unicode 字符。返回值可能并非有效的 Unicode 字符。它与 nil 可进行相等性比较，如果为无效值，则它与 nil 判定为相等（但不可替代 nil）。

如果输入为字符串，则可能返回 nil。如果字符串的第一个字符为有效的 UTF-8 编码的字符，就返回这个字符。

Char 实现的元方法。

Char.__eq
Char.__lt
Char.__le
Char.__tostring

```
Char.is_valid(self) → boolean;
```

Char.is_valid

判断是否为有效的 Unicode 字符。

```
local c = Char('汉');
print(c:is_valid());
local c = Char(0xDFFF);
print(c:is_valid(), c == nil);
.....
true
false true
```

```
Char.from_number(number) → Char;
Char.to_number(self) → number;
```

Char.from_number
Char.to_number

从数字转为 Char 或转为数字。

```
Char.cmp(Char?, Char?) → number | nil;
```

Char.cmp

比较两个字符，如果前者小于后者，返回负数，如果大于后者，返回正数，如果相等，返回 0。如果二者之一为 nil，若是字符无效，返回 0，否则返回 nil。

```
Char.from_string(string) → (Char | nil, boolean);
```

Char.from_string

若字符串开头为无效的 UTF-8 序列，则返回 `nil`，此时第二个返回值总为 `false`。当输入为有效的 UTF-8 字符串时，第一个返回值总是一个有效的 `Char`，当字符串只有一个字符时，第二个返回值为 `true`。

```
print(Char.from_string('汉'));
print(Char.from_string('汉字'));
print(Char.from_string('\xFF'));
.....
汉 true
汉 false
nil false
```

```
Char.chars(string) → function() → Char | nil;
```

Char.chars

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的所有字符。

```
for c in Char.chars('CJK汉字') do
  print(c);
end;
.....
C
J
K
汉
字
```

```
Char.has_han_info(self) → boolean;
Char.han_info(self) → CjkInfo | nil;
```

Char.has_han_info
Char.han_info

判断该字符是否有 CJK 相关的信息，或获取该信息。

Unicode 区块 CJK Unified Ideographs、CJK Extension A ~ J、CJK Compatibility Ideographs、CJK Compatibility Ideographs Supplement 中的所有字符必定有此信息，但“〇”（U+3007）不再此列。

```
print(Char('L'):has_han_info());
print(Char('汉'):has_han_info());
print(Char('〇'):has_han_info()); -- U+3007
.....
false
true
false
```

```
Char.han_ordered_strokes(self) → OrderedStrokes | nil;
```

Char.han_ordered_strokes

返回汉字的笔顺。字形为中国大陆字形。

```
print(Char('L'):han_ordered_strokes());
print(Char('汉'):han_ordered_strokes());
print(Char('〇'):han_ordered_strokes()); -- U + 3007
.....
```

```
nil
44154
nil
```

`Char.script(self) → number;`

`Char.script`

返回字符所属的 Unicode Script 值, `number` 为内部句柄, 可使用 `icu.table.Script` 表查询该值代表的字符串。

```
local icu_data = require("icu.table");
print(icu_data.Script[ Char('L'):script() ]);
print(icu_data.Script[ Char('汉'):script() ]);
print(icu_data.Script[ Char('O'):script() ]);
```

```
.....
Latin
Han
Han
```

`Char.general_category(self) → number;`

`Char.general_category`

返回字符所属的 Unicode General Category 值, `number` 为内部句柄, 可使用 `icu.table.GeneralCategory` 表查询该值代表的字符串。

`Char.is_alphabetic(self) → boolean;`

`Char.is_alphabetic`
`Char.is_numeric`
`Char.is_math`

判断字符是否为字母/数字/数学符号, 范围涵盖所有 Unicode 字符。

`Char.is_ideographic(self) → boolean;`

`Char.is_ideographic`

判断字符是否具有 Unicode Ideographic 属性。不单单只有 CJK 字符有此属性。

`Char.simple_fold(self) → Char;`

`Char.simple_fold`

折叠字符的大小写。字符在折叠大小写之后可进行忽略大小写的比较。

若要对字符串进行忽略大小写的比较, 可使用 `StringRef.eq_ignore_case`。

不可变的 UTF-8 编码的字符串。它并不持有这字符串。

`StringRef`

`StringRef` 实现的元方法。它除了可以与另一个 `StringRef` 进行比较外, 还能与 `string` 进行比较。当它与 `nil` 比较相等时, 仅当它指向空指针时, 被判定为与 `nil` 相等。

`StringRef.__eq`
`StringRef.__lt`
`StringRef.__le`
`StringRef.__len`
`StringRef.__tostring`

`StringRef.is_nil(self) → boolean;`

`StringRef.is_nil`
`StringRef.is_empty`

`StringRef.is_nil` 判断它是否指向空指针, `StringRef.is_empty` 当 `StringRef.is_nil` 为真或其长度为 0 时返回 `true`。

```
StringRef.from_ptr(const StringRef*) → StringRef | nil;
StringRef.unsafe_new(StringRef | string) → StringRef;
StringRef.unsafe_new(const uint8_t*, size_t?) → StringRef;
```

`StringRef.from_ptr`
`StringRef.unsafe_new`

不安全的构建方法。当它指向的值被销毁后, 它本身不一定被销毁, 因此可能出现悬垂引用。

```
StringRef.to_string_escaped(self, all: boolean) → string;
```

StringRef.to_string_escaped

对部分不可见字符转义后输出。如果第二个参数被视为真，则对所有字符都进行转义。每个字符的转义形式为 `\u{HHHH..}`，使用大写的 16 进制。

```
print(StringRef.unsafe_new('\x1F汉OL').to_string_escaped());
print(StringRef.unsafe_new('\x1F汉OL').to_string_escaped(true));
.....
\u{001F}汉OL
\u{001F}\u{6C49}\u{3007}\u{004C}
```

```
StringRef.cmp(StringRef, StringRef) → number;
StringRef.cmp(StringRef, string) → number;
StringRef.cmp(string, StringRef) → number;
```

StringRef.cmp

比较两个字符串。若前者小于后者，返回负数，若等于，返回 0，若大于，返回正数。

```
StringRef.eq_ignore_case(StringRef | string, StringRef | string) → boolean;
StringRef.cmp_ignore_case(StringRef | string, StringRef | string) → number;
```

StringRef.eq_ignore_case
StringRef.cmp_ignore_case

忽略大小写进行比较。若前者小于后者，返回负数，若等于，返回 0，若大于，返回正数。

某些字符在折叠大小写之后可能是多个字符，因此依次对 `Char.simple_fold` 的结果进行比较，其结果并不一定与 `StringRef.eq_ignore_case` 返回的值相等。

```
print(StringRef.eq_ignore_case('Ab汉', 'ab汉'));
print(StringRef.eq_ignore_case('aß', 'ASS'));
print(StringRef.eq_ignore_case('aß', 'AS'));
.....
true
true
false
```

```
StringRef.slice(self, start: number, end_: number?) → StringRef;
```

StringRef.slice

返回从 `start` 到 `end_`（不含）的部分，索引从 0 开始。如果 `end_` 未给出或超过字符串的长度，则该值视为字符串的长度。索引以字节为准，而不是字符。

一般来讲，凡是 `cindex` 的类型，它们的方法使用到索引时，都是从 0 开始，若是需要数字的范围，则都是前闭后开的。

```
print(StringRef.unsafe_new('CJK汉字').slice(1, 6));
.....
JK汉
```

```
StringRef.head(StringRef | string) → number | nil;
```

StringRef.head

返回字符串的第一个字符。

```
print(StringRef.unsafe_new('汉字').head());
.....
27721
```

```
StrRef.chars(self) → function() → number | nil;
```

StrRef.chars

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的所有字符。

```
StrRef.lines(self) → function() → StrRef | nil;
```

StrRef.lines

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的所有行。行结束符不会包含在结果字符串中。

```
local s = "abc\n汉字\n什么";
for line in StrRef.unsafe_new(s):lines() do
    print(line);
end
```

```
.....

abc
汉字
什么
```

```
StrRef.split(self, pattern: nil | Char | StrRef | string)
→ function() → StrRef | nil;
```

StrRef.split

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的子串。分割方式由 **pattern** 指定。

如果模式为 **nil**，则按连续的 ASCII 空白分割，此时，它总不返回空串；否则按指定的字符或字符串分割。如果字符串以 **pattern** 结尾，那么最后一项就是这个 **pattern** 之前的部分子串。

如果输入为空串，则返回的第一个值就是 **nil**。

```
local s = "::std::os::ffi::";
for part in StrRef.unsafe_new(s):split("::") do
    print(part);
end
```

```
.....

std
os
ffi
```

```
local s = " std os ffi \n ";
for part in StrRef.unsafe_new(s):split() do
    print(part);
end
```

```
.....

std
os
s
ffi
```

```
StrRef.graphemes(self) → function() → StrRef | nil;
```

StrRef.graphemes

返回一个迭代器函数，这个函数依次返回每个 **Grapheme Cluster**。

```

local s = "ABC汉字。";
for cluster in StrRef.unsafe_new(s):graphemes() do
    print(cluster);
end

```

```

.....
A
B
C
汉
字
。

```

```
StrRef.unicode_words(self) → function() → StrRef | nil;
```

StrRef.unicode_words

返回一个迭代器函数，这个函数依次返回每个单词。

```
StrRef.split_word_bounds(self) → function() → StrRef | nil;
```

StrRef.split_word_bounds

返回一个迭代器函数，这个函数返回一个根据 **Word Boundaries** 分隔开来的子串。

```

local s = [[ABC: ( “汉字” )。]];
for word in StrRef.unsafe_new(s):unicode_words() do
    print(word);
end

```

```

.....
ABC
汉
字

```

```

local s = [[ABC: ( “汉字” )。]];
for part in StrRef.unsafe_new(s):split_word_bounds() do
    print(part);
end

```

```

.....
ABC
:
(
“
汉
字
”
)
。

```

```

StrRef.trim_ascii_spaces() → StrRef | nil;
StrRef.trim_ascii_spaces(start: boolean) → StrRef | nil;
StrRef.trim_ascii_spaces(start: boolean, end: boolean) → StrRef | nil;

```

StrRef.trim_ascii_spaces

返回删除字符串前后空格的结果。如果是有效的字符串，不会返回 **nil**。默认情况下，前后的空格都被删除。

```
StrRef.is_emoji(self) → boolean;
```

StrRef.is_emoji

判断字符串是否为 Emoji 序列，有的 Emoji 是由多个 Unicode 字符组成。

```
print("#😄", StrRef.unsafe_new("😄"):is_emoji());
print("#🇨🇳", StrRef.unsafe_new("🇨🇳"):is_emoji());
print("#🇨🇳", StrRef.unsafe_new("🇨🇳"):is_emoji());
print("#🇨🇳", StrRef.unsafe_new("🇨🇳"):is_emoji());
print("#汉", StrRef.unsafe_new("汉"):is_emoji());
print("#L", StrRef.unsafe_new("L"):is_emoji());
```

```
.....
4 true
8 true
17 true
35 true
3 false
1 false
```

保存 CJK 相关的信息。这个类型拥有其数据。一般通过 `Char.han_info` 获取。

CjkInfo

实现的元方法。

CjkInfo.__eq
CjkInfo.__tostring

```
CjkInfo.radical_from_kind(kind: StrRef | string) → number | nil;
```

CjkInfo.radical_from_kind

从康熙部首的种类返回其康熙部首的内部表示。种类是 Unicode `CJKRadicals.txt` 数据的第一列，一般是 1~214 的数字，有的还可带 1 至 3 个西文单引号，如 `123`, `213''` 分别表示第 123 个康熙部首，以及第 213 个康熙部首的第 2 个变体。

```
print(true)
.....
```

```
CjkInfo.is_nil(self) → boolean;
```

CjkInfo.is_nil

判断是否为无效值。

```
CjkInfo.contains_char(Into<Char>) → boolean;
```

CjkInfo.contains_char

判断字符是否有 CJK 信息。它与 `Char.has_han_info` 完全一致。

```
print(true)
.....
```

```
CjkInfo.kx_radical_char(self) → Char;
```

CjkInfo.kx_radical_char

返回康熙部首的具体字符，这字符位于 Unicode 的康熙部首相关区块。某些康熙部首的变体在康熙部首相关区块中没有对应值，因此，返回的字符可能无效。

```
CjkInfo.kx_ideography_char(self) → Char;
```

CjkInfo.kx_ideography_char

返回康熙部首的具体字符，这字符位于 Unicode 的 CJK 表意字相关区块。所有康熙部首及其变体在表意字区块都有对应值，因此有效的 `CjkInfo` 总返回有效的字符。

```
CjkInfo.kangxi_radical_number(self) → number | nil;
```

CjkInfo.kangxi_radical_number

返回康熙部首的号码，目前的返回值为 1 ~ 214。

```
CjkInfo.kangxi_radical_ideography_slot(self)  
→ number | nil;
```

CjkInfo.kangxi_radical_ideography_slot

返回康熙部首在 CJK 表意字区块中的对应值的代码点。康熙部首的变体被映射为原始的 214 个部首。

```
CjkInfo.kx_radical_kind(self) → StrRef | nil;
```

CjkInfo.kx_radical_kind

返回康熙部首的字符串表示。由数字和末尾的 03 个 ' 组成。

```
CjkInfo.ucd_additional_strokes(self) → number;
```

CjkInfo.ucd_additional_strokes

返回 Unicode 数据中该字符除部首外的笔画数，该值可能为负数。

```
CjkInfo.ucd_total_strokes(self) → number;
```

CjkInfo.ucd_total_strokes

返回 Unicode 数据中该字符的总笔画数。这与所选字形无关，另见 **CjkInfo.total_strokes**。

```
CjkInfo.total_strokes(self) → number;
```

CjkInfo.total_strokes

返回字符的总笔画数。不同字形标准，总笔画数可能不同，该函数使用中国大陆字形标准。它的值不一定和 **CjkInfo.ucd_total_strokes** 返回的一致。

```
CjkInfo.original_radical(self) → number | nil;
```

CjkInfo.original_radical

如果是康熙部首的变体，则返回其原始的康熙部首的内部表示，若就是原始部首，则返回原始部首的内部表示。对于有效的内部值，总是不返回 nil。

```
CjkInfo.mandarin(self) → nil | (StrRef, number);
```

CjkInfo.mandarin

如果该字符有拼音信息，第一个返回值是它最常见的拼音，第二个返回值是它的声调。

```
CjkInfo.cantonese(self) → nil | (StrRef, number);
```

CjkInfo.cantonese

如果该字符有粤语拼音信息，第一个返回值是它最常见的拼音，第二个返回值是它的声调。

```
Emoji(string | StrRef) → Emoji | nil;
```

Emoji

如果字符串是一个有效的 Emoji 序列，则返回这个 Emoji，Emoji 类型是 Owned。

把 emoji 转为字符串。

Emoji.__tostring

```
Emoji.name(self) → StrRef | nil;
```

Emoji.name

返回 emoji 的名称。一般情况下该方法不会返回 nil。

```
Emoji.group(self) → number;
```

Emoji.group

返回 emoji 组别的内部表示。可使用 **icu.table.EmojiGroup** 获取组别的名字。

汉字的笔画。

OrderedStrokes

OrderedStrokes 实现的元方法。__ipairs 返回的索引号从 1 开始。

`OrderedStrokes.__len`
`OrderedStrokes.__eq`
`OrderedStrokes.__lt`
`OrderedStrokes.__gt`
`OrderedStrokes.__ipairs`

转为字符串的形式。字符串的每一项都是 1~5 的某个数字，按顺序分别代表：横、竖、撇、点（捺）、折（弯钩）。

`OrderedStrokes.__toString`

```
OrderedStrokes[number] → number;  
OrderedStrokes.get(n: number) → number;
```

`OrderedStrokes.__index`
`OrderedStrokes.get`

获取第 n 画。 n 从 0 开始。数字为 1~5 的某个数字，按顺序分别代表：横、竖、撇、点（捺）、折（弯钩）。

1.2 索引条目

未合并的索引。

IndexEntryRef

IndexEntryRef 可以与 `nil` 进行比较，当其为无效值时，被视为与 `nil` 相等。

`IndexEntryRef.__eq`

```
IndexEntryRef.is_nil(self) → boolean;
```

`IndexEntryRef.is_nil`

判断是否为 `nil`。

```
IndexEntryRef.to_handle(self) → number;
```

`IndexEntryRef.to_handle`

返回一个句柄。一般来讲，**IndexEntryRef** 是不能作为数字的索引的，可转为句柄。

```
IndexEntryRef.levels_count(self) → number;
```

`IndexEntryRef.levels_count`

返回此条目的层级数。

```
IndexEntryRef.key_n(self, n: number) → StrRef | nil;  
IndexEntryRef.keys(self) → function() → StrRef | nil;
```

`IndexEntryRef.key_n`
`IndexEntryRef.keys`

返回该条目第 n 个排序关键字， n 从 0 开始。或返回一个迭代器函数。

```
IndexEntryRef.as_table(self) → table;
table := {
  levels: { { key?: string, display: string }, .. },
  range?: "None" | "Open" | "Close",
  page_commands?: string,
  pages?: { { kind: string, value: number }, .. },
  pages_raw?: string,
}
```

IndexEntryRef.as_table

把条目表示为 lua 表。当前 kind 的值有：

- Arabic (阿拉伯数字)，
- alpha (小写拉丁字母组成的 26 进制数字)，
- Alpha (大写拉丁字母组成的 26 进制数字)，
- roman (小写罗马数字)，
- Roman (大写罗马数字)，
- ZhDigits (中文数串，由“零〇一二三四五六七八九”组成)，
- ZhNumber (中文数字)。

```
IndexEntryRef.is_same(lhs, rhs) → boolean;
```

IndexEntryRef.is_same

判断两个条目的排序关键字是否完全相等。

存储索引条目的数组。

IndexEntryArray

IndexEntryArray 实现的元方法。__ipairs 返回的索引数字从 1 开始。

IndexEntryArray.__len
IndexEntryArray.__ipairs

```
IndexEntryArray.is_nil(self) → boolean;
IndexEntryArray.is_empty(self) → boolean;
```

IndexEntryArray.is_nil
IndexEntryArray.is_empty

判断数组是否为无效值，或其不含任何值。

```
IndexEntryArray[number] → IndexEntryRef;
IndexEntryArray.get(self, index: number) → IndexEntryRef;
```

IndexEntryArray.__index
IndexEntryArray.get

用数字索引数组，索引从 0 开始。若索引大于或等于数组的长度，**IndexEntryRef** 并非一个有效值，可使用 == nil 或 **IndexEntryRef.is_nil** 来判断是否有效。

```
IndexEntryArray.push(self, IndexEntryRefMut);
IndexEntryArray.push(self, table);
```

IndexEntryArray.push

在数组末尾附加一个条目。table 必须能转为合法的 **IndexEntryRefMut**，表的结构与 **IndexEntryRef.as_table** 返回的一致。

```
IndexEntryArray.extend_from_iterator(self, init);
init := function() →
    function() → IndexEntryRefMut | table | nil;
```

IndexEntryArray.extend_from_iterator

把迭代器返回的每一项附加到数组中。init 返回一个迭代器函数。

```

IndexEntryRefMut() → IndexEntryRefMut;
IndexEntryRefMut(table) → IndexEntryRefMut;
IndexEntryRefMut(IndexEntryRef) → IndexEntryRefMut;
IndexEntryRefMut(IndexEntryRefMut) → IndexEntryRefMut;

```

IndexEntryRefMut

IndexEntryRefMut 的构造函数。它创建一个新的空索引条目，或从传入的参数中构造出一个新的索引条目。**table** 的结构和 **IndexEntryRef.as_table** 返回的一致。

```

IndexEntryRefMut.push_level(self, key: nil, display: string | StrRef)
    → boolean;
IndexEntryRefMut.push_level(self, level: string | StrRef)
    → boolean;
IndexEntryRefMut.push_level(self, key: string, display: string)
    → boolean;
IndexEntryRefMut.push_level(self, key: StrRef, display: StrRef)
    → boolean;

```

IndexEntryRefMut.push_level

在条目中插入一个层级。

```

IndexEntryRefMut.set_range(self, range?: StrRef | string) → boolean;

```

IndexEntryRefMut.set_range

设置本条目是否是页码范围的起始或结束位置。**range** 可选值为 **None**、**Open**、**Close**，如果不设置，则相当于 **None**。返回值表示是否成功设置。

```

IndexEntryRefMut.set_page_commands(
    self,
    page_commands?: StrRef | string
) → boolean;

```

IndexEntryRefMut.set_page_commands

设置本条目页码要使用的命令。如果未给出或为 **nil**，相当于清除已经设置的命令。

```

IndexEntryRefMut.push_page(
    self,
    kind: StrRef | string,
    value: Into<number>
) → boolean;

```

IndexEntryRefMut.push_page

向本条目中插入一个页码段。页码种类如 **IndexEntryRef.as_table** 所列。

一个条目能同时使用多级页码，如 MMMI-2026-c-8，就依次使用了 **Roman**、**Arabic**、**alpha**、**Arabic** 四级页码，但只算作一个页码。

```

IndexEntryRefMut.set_pages_raw(self, pages_raw: StrRef | string)
    → boolean;

```

IndexEntryRefMut.set_pages_raw

设置页码的显示效果。在写入到输出文件时，总是使用这个值，页码的具体数值只用于排序等情形，因此它可以不和具体的值挂钩。

```

IndexEntryRefMut.as_table(self) → table;

```

IndexEntryRefMut.as_table

转为 lua 表，它本身的值保持不变。表的结构与 **IndexEntryRef.as_table** 返回的一致。

合并后的索引条目。

MergedEntryRef

```
MergedEntryRef.is_nil(self) → boolean;
MergedEntryRef.to_handle(self) → number;
```

```
MergedEntryRef._eq
MergedEntryRef.is_nil
MergedEntryRef.to_handle
```

MergedEntryRef.is_nil 用于判断是否为无效值。**MergedEntryRef.to_handle** 用于把条目转为内部句柄。

```
MergedEntryRef.levels_count(self) → number;
MergedEntryRef.level_n(self, n: number) → StrRef | nil;
MergedEntryRef.levels(self)
    → function() → StrRef | nil;
```

```
MergedEntryRef.levels_count
MergedEntryRef.level_n
MergedEntryRef.levels
```

返回条目的层级数，或返回对应的层级。

```
MergedEntryRef.key_n(self, n: number) → StrRef | nil;
MergedEntryRef.keys(self) → function() → StrRef | nil;
```

```
MergedEntryRef.key_n
MergedEntryRef.keys
```

返回条目的排序关键字。关键字的个数个层级数一致。

```
MergedEntryRef.pages_count(self) → number;
```

```
MergedEntryRef.pages_count
```

返回合并后的索引条目包含的页码（或页码区间）数量。

```
MergedEntryRef.page_command_n(self, n: number) → StrRef | false | nil;
MergedEntryRef.page_commands(self) → function() → StrRef | false | nil;
```

```
MergedEntryRef.page_command_n
MergedEntryRef.page_commands
```

返回合并后的索引条目的页码在输出时要使用的命令。如果页码没有设置命令，返回 **false**。

```
MergedEntryRef.pages_raw_n(self, n: number)
    → nil | StrRef | (StrRef, StrRef);
MergedEntryRef.pages_raws(self)
    → function() → nil | StrRef | (StrRef, StrRef);
```

```
MergedEntryRef.pages_raw_n
MergedEntryRef.pages_raw
```

返回页码要输出的原始字符串。如果是单个页码则返回一个 **StrRef**，如果是页码区间，则返回两个 **StrRef**。

```
MergedEntryRef.page_span_n(self, n: number) → number;
MergedEntryRef.page_spans(self) → function() → number | nil;
```

```
MergedEntryRef.page_span_n
MergedEntryRef.page_spans
```

返回页码区间的长度。如果是无效页码，返回 0，如果是单个页码，返回 1，否则返回区间的长度。

```
MergedEntryRef.max_same_key(MergedEntryRef, MergedEntryRef) → number;
```

```
MergedEntryRef.max_same_key
```

返回两个条目拥有的相同排序键的个数。

```
MergedEntryRef.as_table(self) → table;
table := {
  levels: { { key?: string, display: string }, .. },
  pages?: { page_entry, .. },
}
page_entry := {
  kind: "single",
  page_command: nil | string,
  pages: { { kind: string, value: number }, .. },
  pages_raw: string,
} | {
  kind: "range",
  page_command: nil | string,
  start: { { kind: string, value: number }, .. },
  start_raw: string,
  end: { { kind: string, value: number }, .. },
  end_raw: string,
}
```

MergedEntryRef.as_table

以 lua 表的形式表示该条目。页码的种类见 [IndexEntryRef.as_table](#)。

1.3 实用类型

```
Logger.error(string | StrRef) → boolean;
```

在日志中写入对应级别的信息。

Logger.error
Logger.warn
Logger.info
Logger.debug
Logger.trace

```
IstInputStyle: const table;
```

索引的输入格式。

IstInputStyle

```
IstInputStyle.is_nil(self) → boolean;
```

判断是否不存在任何输入格式。一般情况下，输入格式都是存在的。

IstInputStyle.is_nil

```
IstInputStyle.keyword() → StrRef | nil;
```

返回输入格式对应关键字的值。

IstInputStyle.keyword
IstInputStyle.arg_open
IstInputStyle.arg_close
IstInputStyle.range_open
IstInputStyle.range_close
IstInputStyle.level
IstInputStyle.actual
IstInputStyle.encap
IstInputStyle.quote
IstInputStyle.escape
IstInputStyle.page_compositor
IstInputStyle.comment
IstInputStyle.separator

```
IstOutputStyle: const table;
```

索引的输出格式。

IstOutputStyle

```
IstOutputStyle.is_nil(self) → boolean;
```

判断是否不存在任何输出格式。一般情况下，输出格式都是存在的。

IstOutputStyle.is_nil

```
IstOutputStyle.preamble() → StrRef | nil;
```

```
IstOutputStyle.preamble  
IstOutputStyle.postamble  
IstOutputStyle.group_skip  
IstOutputStyle.heading_prefix  
IstOutputStyle.heading_suffix  
IstOutputStyle.headings_flag  
IstOutputStyle.numhead_positive  
IstOutputStyle.numhead_negative  
IstOutputStyle.symhead_positive  
IstOutputStyle.symhead_negative  
IstOutputStyle.item_0  
IstOutputStyle.item_1  
IstOutputStyle.item_2  
IstOutputStyle.item_01  
IstOutputStyle.item_x1  
IstOutputStyle.item_12  
IstOutputStyle.item_x2  
IstOutputStyle.delim_0  
IstOutputStyle.delim_1  
IstOutputStyle.delim_2  
IstOutputStyle.delim_n  
IstOutputStyle.delim_r  
IstOutputStyle.delim_t  
IstOutputStyle.encap_prefix  
IstOutputStyle.encap_infix  
IstOutputStyle.encap_suffix  
IstOutputStyle.page_precedence  
IstOutputStyle.suffix_2p  
IstOutputStyle.suffix_3p  
IstOutputStyle.suffix_mp  
IstOutputStyle.stroke_prefix  
IstOutputStyle.stroke_suffix  
IstOutputStyle.radical_prefix  
IstOutputStyle.radical_suffix  
IstOutputStyle.radical_simplified_flag  
IstOutputStyle.radical_simplified_prefix  
IstOutputStyle.radical_simplified_delimiter  
IstOutputStyle.radical_simplified_suffix
```

返回输出格式对应关键字的值。

```
CliOptions: const {  
  c: boolean,  
  q: boolean,  
  r: boolean,  
  strict: boolean,  
  z?: string,  
  extra_options?: { string, .. },  
};
```

CliOptions

命令行选项。extra_options 是命令行参数 -- 之后的内容。

```
CIndex.Util: const table;  
CIndex.Ideographic: const table;  
CIndex.Entry: const table;
```

CIndex.Util
CIndex.Ideographic
CIndex.Entry

cindex 提供的实用模块。

```
CIndex.Util.is_ctype(type, value) → boolean;
```

CIndex.Util.is_ctype

判断某个值是否为某个 cindex 类型。无法判别所有 lua 原生类型。

例如，CIndex.Ideographic.is_ctype(StrRef, s) 即判断 s 是否为 StrRef 类型。

```
CIndex.Util.table_to_json_string(table) → string;
```

```
CIndex.Util.table_to_json_string
```

把表转为 JSON 字符串。

```
CIndex.Util.value_from_json_string(string) → ??;
```

```
CIndex.Util.value_from_json_string
```

把 JSON 字符串转为某个 lua 基本类型。

```
CIndex.Ideographic.group_by_pinyin(Char) → string;
```

```
CIndex.Ideographic.group_by_pinyin
```

```
CIndex.Ideographic.group_by_bihua
```

```
CIndex.Ideographic.group_by_bushou
```

返回字符按拼音/笔画数/部首的组别，组别见 `cindex.table.GroupOrder`。

如果字符不位于 CJK 表意字相关区块或没有对应信息，则返回 `Symbols`，即认为它是符号组。

除了 CJK 表意字相关区块的字符外，还识别表 1 补充的字符的信息。

字符	代码点	拼音	部首	笔顺
〇	U+3007	líng	乙 (5)	5

表 1: 补充的字符信息

```
CIndex.Entry.compare_by_default(MergedEntryRef, MergedEntryRef)
→ boolean;
```

```
CIndex.Entry.compare_by_default
```

```
CIndex.Entry.compare_by_pinyin
```

```
CIndex.Entry.compare_by_bihua
```

```
CIndex.Entry.compare_by_bushou
```

根据指定的比较算法比较两个条目，如果前者应当排在后者之前，则返回 `true`。

总体来说，`cindex` 首先进行忽略大小写的比较，若相等，再区分大小写进行比较，顺序按照 Unicode 的顺序。若汉字与非汉字进行比较，总是非汉字排在前面，而不考虑它们在 Unicode 中的顺序。

使用读音排序时，没有读音数据的字符（包括生僻字）一律排在有读音的字符之前，汉字按其最常用读音比较，读音相同汉字的按 Unicode 编码排序。

使用笔画数和笔顺排序时，笔画数小的排在前面，笔画数相同的，按横、竖、撇、点（捺）、折的顺序逐笔画比较，仍然相同的按 Unicode 编码排序；生僻汉字没有笔顺信息的，排在同笔画数有笔顺的字后面。

使用部首和除部首笔画数排序时，部首按康熙字典 214 部首顺序排列，部首和笔画数相同的按 Unicode 编码排序。

1.4 PCRE2 正则表达式

PCRE2 是 C 语言实现正则表达式库。关于它的相关信息和语法，见 <https://pcre2project.github.io/pcre2/doc/>。

`cindex` 对其做了简单封装。

```
Regex(string | StrRef | const uint8_t*, options: number) → Regex;
```

```
Regex
```

创建一个正则表达式。无效的模式直接报错。

```
Regex.jit_compile(self) → true | (false, string);
```

```
Regex.jit_compile
```

尝试使用 JIT 编译。返回是否成功。如果为 `false`，还返回相关错误码。

```
Regex.is_match(self, subject: StrRef | string) → boolean;
```

```
Regex.is_match
```

检查 `subject` 是否匹配正则表达式。

```
Regex.find(self, subject: StrRef, start_pos: number) → StrRef | nil;
Regex.find(self, subject: string, start_pos: number) → string | nil;
```

Regex.find

从字符串的 `start_pos` 位置开始，查找匹配该的正则表达式的子串并返回。不论是 `StrRef` 还是 `string`，索引均从 0 开始。

```
Regex.captures(self, subject: StrRef, start_pos: number)
    → nil | { [number] = StrRef, [string] = StrRef, .. };
Regex.captures(self, subject: string, start_pos: number)
    → nil | { [number] = string, [string] = string, .. };
```

Regex.captures

查找并返回字符串的捕获组。第 0 个索引是匹配的子串。

```
Regex.gmatch(self, subject: StrRef, start_pos: number)
    → nil | { [number] = StrRef, [string] = StrRef, .. };
Regex.gmatch(self, subject: string, start_pos: number)
    → nil | { [number] = string, [string] = string, .. };
```

Regex.gmatch

返回一个迭代器函数，其返回值依次是匹配的捕获。

```
Regex.gsub(
    self,
    subject: StrRef | string,
    replacement: StrRef | string,
    options?: number,
    once?: boolean;
) → string;
```

Regex.gsub

以正则表达式 `replacement` 替换 `subject` 中匹配的内容。

```
Regex.Constant: const { [string] = number, .. };
```

Regex.Constant

这个表保存一些 PCRE2 定义的常量。

```
Regex.Constant.UNSET: number;
```

Regex.Constant.UNSET
Regex.Constant.ZERO_TERMINATED

Regex.Constant.ALLOW_EMPTY_CLASS: **number**;

Regex.Constant.ALLOW_EMPTY_CLASS
Regex.Constant.ALT_BSUX
Regex.Constant.AUTO_CALLOUT
Regex.Constant.CASELESS
Regex.Constant.DOLLAR_ENDONLY
Regex.Constant.DOTALL
Regex.Constant.DUPNAMES
Regex.Constant.EXTENDED
Regex.Constant.FIRSTLINE
Regex.Constant.MATCH_UNSET_BACKREF
Regex.Constant.MULTILINE
Regex.Constant.NEVER_UCP
Regex.Constant.NEVER_UTF
Regex.Constant.NO_AUTO_CAPTURE
Regex.Constant.NO_AUTO_POSSESS
Regex.Constant.NO_DOTSTAR_ANCHOR
Regex.Constant.NO_START_OPTIMIZE
Regex.Constant.UCP
Regex.Constant.UNGREEDY
Regex.Constant.UTF
Regex.Constant.NEVER_BACKSLASH_C
Regex.Constant.ALT_CIRCUMFLEX
Regex.Constant.ALT_VERBNAMES
Regex.Constant.USE_OFFSET_LIMIT
Regex.Constant.EXTENDED_MORE
Regex.Constant.LITERAL
Regex.Constant.MATCH_INVALID_UTF
Regex.Constant.ALT_EXTENDED_CLASS

Regex.Constant.NOTBOL: **number**;

Regex.Constant.NOTBOL
Regex.Constant.NOTEOL
Regex.Constant.NOTEMPTY
Regex.Constant.NOTEMPTY_ATSTART
Regex.Constant.PARTIAL_SOFT
Regex.Constant.PARTIAL_HARD
Regex.Constant.DFA_RESTART
Regex.Constant.DFA_SHORTEST
Regex.Constant.SUBSTITUTE_GLOBAL
Regex.Constant.SUBSTITUTE_EXTENDED
Regex.Constant.SUBSTITUTE_UNSET_EMPTY
Regex.Constant.SUBSTITUTE_UNKNOWN_UNSET
Regex.Constant.SUBSTITUTE_OVERFLOW_LENGTH
Regex.Constant.NO_JIT
Regex.Constant.COPY_MATCHED_SUBJECT
Regex.Constant.SUBSTITUTE_LITERAL
Regex.Constant.SUBSTITUTE_MATCHED
Regex.Constant.SUBSTITUTE_REPLACEMENT_ONLY

```
Regex.Constant.INFO_ALLOPTIONS: number;
```

```
Regex.Constant.INFO_ALLOPTIONS
Regex.Constant.INFO_ARGOPTIONS
Regex.Constant.INFO_BACKREFMAX
Regex.Constant.INFO_BSR
Regex.Constant.INFO_CAPTURECOUNT
Regex.Constant.INFO_FIRSTCODEUNIT
Regex.Constant.INFO_FIRSTCODETYPE
Regex.Constant.INFO_FIRSTBITMAP
Regex.Constant.INFO_HASCORRLF
Regex.Constant.INFO_JCHANGED
Regex.Constant.INFO_JITSIZE
Regex.Constant.INFO_LASTCODEUNIT
Regex.Constant.INFO_LASTCODETYPE
Regex.Constant.INFO_MATCHEMPTY
Regex.Constant.INFO_MATCHLIMIT
Regex.Constant.INFO_MAXLOOKBEHIND
Regex.Constant.INFO_MINLENGTH
Regex.Constant.INFO_NAMECOUNT
Regex.Constant.INFO_NAMEENTRYSIZE
Regex.Constant.INFO_NAMETABLE
Regex.Constant.INFO_NEWLINE
Regex.Constant.INFO_DEPTHLIMIT
Regex.Constant.INFO_RECURSIONLIMIT
Regex.Constant.INFO_SIZE
Regex.Constant.INFO_HASBACKSLASHC
Regex.Constant.INFO_FRAME_SIZE
Regex.Constant.INFO_HEAPLIMIT
Regex.Constant.INFO_EXTRAOPTIONS
```

§ 2 可加载的外部库

cindex 对加载外部 lua 文件和库做了限制，目前只允许加载某些特定的库。

2.1 cindex.table

cindex 的相关表。可使用 `require("cindex.table")` 加载。

cindex.table

```
cindex.table.GroupOrder: const {
  "nil" = 0,
  "Symbols" = ??,
  "Numbers" = ??,
  "A" = ??, .., "Z" = ??,
  "BiHua1" = ??, .., "BiHua64" = ??,
  "BuShou1" = ??, .., "BuShou214" = ??,
  [number] = string, ..
};
```

cindex.table.GroupOrder

cindex 默认的各个组别及其排序。Symbols 是符号分组，Numbers 是数字分组，BiHua.. 是笔画数分组，BuShou.. 是康熙部首分组。每个分组对应一个数字。也可通过数字反查对应的组别名。

```
BiHuaToGroup: const { [1] = string, .., [64] = string };
```

cindex.table.BiHuaToGroup

根据笔画数返回组别名。

```
BuShouToGroup: const { [number] = string, .. }
```

cindex.table.BuShouToGroup

根据康熙部首的代码点（包括康熙部首相关区块和表意字相关区块）值返回组别名。

```
BuShouToChars: const {
  [1] = { number, .. }, .., [214] = { number, .. },
  ["BuShou1"] = { number, .. }, .., ["BuShou214"] = { number, .. },
};
```

`cindex.table.BuShouToChars`

康熙部首对应的表意字字符，如果有变体，则依次存储在对应的表中。

2.2 icu.table

`icu.table` 保存 Unicode 相关的信息。可使用 `require("icu.table")` 加载。

`icu.table`

```
Script = { [number] = string, [string] = number, .. }
```

`icu.table.Script`

`Char.script` 返回的句柄可用于索引此表。也可通过字符串反查对应的句柄。

```
GeneralCategory = { [number] = string, [string] = number, .. }
```

`icu.table.GeneralCategory`

`Char.general_category` 返回的句柄可用于索引此表。也可通过字符串反查对应的句柄。

```
EmojiGroup = { [number] = group, [string] = number, .. }
group := |
  "Activities"
  "Animals And Nature"
  "Flags"
  "Food And Drink"
  "Objects"
  "People And Body"
  "Smileys And Emotion"
  "Symbols"
  "Travel And Places"
  ??
```

`icu.table.EmojiGroup`

Emoji 的分组。目前共有 9 个组别。

```
PropertyShortNames = { [string] = string, .. }
```

`icu.table.PropertyShortNames`

属性值对应的短名。

2.3 digest 哈希算法

2.4 LPeg

LPeg 是基于 Parsing Expression Grammars 的模式匹配库。`cindex` 内置了此库，可使用 `require("lpeg")` 加载。详细用法见 <https://www.inf.puc-rio.br/~roberto/lpeg/lpeg.html>。

§ 3 代码索引

粗体的数字表示描述对应索引项的页码；意大利体的数字表示使用对应索引项的页码。

L

lua 类型

__ipairs	9
Char	1, 1, 2
__eq	1
__le	1
__lt	1
__tostring	1
chars	2
cmp	1
from_number	1
from_string	2
general_category	3, 19
han_info	2, 7
han_ordered_strokes	2
has_han_info	2, 7
is_alphabetic	3
is_ideographic	3
is_math	3
is_numeric	3
is_valid	1
script	3, 19
simple_fold	3, 4
to_number	1
CIndex	
Entry	14
Ideographic	14
Util	14
CIndex.Entry	
compare_by_bihua	15
compare_by_bushou	15
compare_by_default	15
compare_by_pinyin	15
CIndex.Ideographic	
group_by_bihua	15
group_by_bushou	15
group_by_pinyin	15
cindex.table	18
BiHuaToGroup	18
BuShouToChars	19
BuShouToGroup	18
GroupOrder	15, 18
CIndex.Util	
is_ctype	14
table_to_json_string	15

value_from_json_string	15
CjkInfo	7, 7
__eq	7
__tostring	7
cantonese	8
contains_char	7
is_nil	7
kangxi_radical_ideography_slot	8
kangxi_radical_number	8
kx_ideography_char	7
kx_radical_char	7
kx_radical_kind	8
mandarin	8
original_radical	8
radical_from_kind	7
total_strokes	8, 8
ucd_additional_strokes	8
ucd_total_strokes	8, 8
CliOptions	14
Emoji	8
__tostring	8
group	8
name	8
icu.table	19, 19
EmojiGroup	8, 19
GeneralCategory	3, 19
PropertyShortNames	19
Script	3, 19
IndexEntryArray	10, 10
__index	10
__ipairs	10
__len	10
extend_from_iterator	10
get	10
is_empty	10
is_nil	10
push	10
IndexEntryRef	9, 9, 10
__eq	9
as_table	10, 10, 11, 13
is_nil	9, 10
is_same	10
key_n	9
keys	9
levels_count	9

to_handle	9	preamble	14
IndexEntryRefMut	10, 11, 11	radical_prefix	14
as_table	11	radical_simplified_delimiter	14
push_level	11	radical_simplified_flag	14
push_page	11	radical_simplified_prefix	14
set_page_commands	11	radical_simplified_suffix	14
set_pages_raw	11	radical_suffix	14
set_range	11	stroke_prefix	14
IstInputStyle	13	stroke_suffix	14
actual	13	suffix_2p	14
arg_close	13	suffix_3p	14
arg_open	13	suffix_mp	14
comment	13	symhead_negative	14
encap	13	symhead_positive	14
escape	13	Logger	
is_nil	13	debug	13
keyword	13	error	13
level	13	info	13
page_compositor	13	trace	13
quote	13	warn	13
range_close	13	MergedEntryRef	11
range_open	13	__eq	12
separator	13	as_table	13
IstOutputStyle	13	is_nil	12, 12
delim_0	14	key_n	12
delim_1	14	keys	12
delim_2	14	level_n	12
delim_n	14	levels	12
delim_r	14	levels_count	12
delim_t	14	max_same_key	12
encap_infix	14	page_command_n	12
encap_prefix	14	page_commands	12
encap_suffix	14	page_span_n	12
group_skip	14	page_spans	12
heading_prefix	14	pages_count	12
heading_suffix	14	pages_raw	12
headings_flag	14	pages_raw_n	12
is_nil	13	to_handle	12, 12
item_0	14	OrderedStrokes	9, 9
item_01	14	__eq	9
item_1	14	__gt	9
item_12	14	__index	9
item_2	14	__ipairs	9
item_x1	14	__len	9
item_x2	14	__lt	9
numhead_negative	14	__tostring	9
numhead_positive	14	get	9
page_precedence	14	Regex	15
postamble	14	captures	16

Constant	16	INFO_RECURSIONLIMIT	18
find	16	INFO_SIZE	18
gmatch	16	LITERAL	17
gsub	16	MATCH_INVALID_UTF	17
is_match	15	MATCH_UNSET_BACKREF	17
jit_compile	15	MULTILINE	17
Regex.Constant		NEVER_BACKSLASH_C	17
ALLOW_EMPTY_CLASS	17	NEVER_UCP	17
ALT_BSUX	17	NEVER_UTF	17
ALT_CIRCUMFLEX	17	NO_AUTO_CAPTURE	17
ALT_EXTENDED_CLASS	17	NO_AUTO_POSSESS	17
ALT_VERBNAMES	17	NO_DOTSTAR_ANCHOR	17
AUTO_CALLOUT	17	NO_JIT	17
CASELESS	17	NO_START_OPTIMIZE	17
COPY_MATCHED_SUBJECT	17	NOTBOL	17
DFA_RESTART	17	NOTEMPTY	17
DFA_SHORTEST	17	NOTEMPTY_ATSTART	17
DOLLAR_ENDONLY	17	NOTEOL	17
DOTALL	17	PARTIAL_HARD	17
DUPNAMES	17	PARTIAL_SOFT	17
EXTENDED	17	SUBSTITUTE_EXTENDED	17
EXTENDED_MORE	17	SUBSTITUTE_GLOBAL	17
FIRSTLINE	17	SUBSTITUTE_LITERAL	17
INFO_ALLOPTIONS	18	SUBSTITUTE_MATCHED	17
INFO_ARGOPTIONS	18	SUBSTITUTE_OVERFLOW_LENGTH	17
INFO_BACKREFMAX	18	SUBSTITUTE_REPLACEMENT_ONLY	17
INFO_BSR	18	SUBSTITUTE_UNKNOWN_UNSET	17
INFO_CAPTURECOUNT	18	SUBSTITUTE_UNSET_EMPTY	17
INFO_DEPTHLIMIT	18	UCP	17
INFO_EXTRAOPTIONS	18	UNGREEDY	17
INFO_FIRSTBITMAP	18	UNSET	16
INFO_FIRSTCODETYPE	18	USE_OFFSET_LIMIT	17
INFO_FIRSTCODEUNIT	18	UTF	17
INFO_FRAME_SIZE	18	ZERO_TERMINATED	16
INFO_HASBACKSLASHC	18	StrRef	3, 3, 12, 14, 16
INFO_HASCORRLF	18	__eq	3
INFO_HEAPLIMIT	18	__le	3
INFO_JCHANGED	18	__len	3
INFO_JIT_SIZE	18	__lt	3
INFO_LASTCODETYPE	18	__tostring	3
INFO_LASTCODEUNIT	18	chars	5
INFO_MATCHEMPTY	18	cmp	4
INFO_MATCHLIMIT	18	cmp_ignore_case	4
INFO_MAXLOOKBEHIND	18	eq_ignore_case	3, 4, 4
INFO_MINLENGTH	18	from_ptr	3
INFO_NAMECOUNT	18	graphemes	5
INFO_NAMEENTRY_SIZE	18	head	4
INFO_NAMETABLE	18	is_emoji	7
INFO_NEWLINE	18	is_empty	3, 3

is_nil.....	3, 3	to_string_escaped	4
lines	5	trim_ascii_spaces	6
slice	4	unicode_words	6
split	5	unsafe_new	3
split_word_bounds	6		